UDACITY

Part of **Accenture**

SCHOOL OF PROGRAMMING AND DEVELOPMENT

# C++

# Syllabus

UDACITY.COM

# Overview

Learn C++, a high-performance programming language used in the world's most exciting engineering jobs -- from self-driving cars and robotics to web browsers, media platforms, servers, and video games.

Nanodegree Program | Intermediate | 42 hours | 4.6 (812 Reviews)

# Prerequisites

Prior to enrolling, you should have the following knowledge:

Basic C++ | C++ control flow | C++ functions | Basic Linux | Intermediate computer programming

You will also need to be able to communicate fluently and professionally **in written and spoken English**.

# Skills You'll Learn

## Introduction to C++

A* Search Algorithm | C++ syntax | C++ data structures | C++ file handling | C++ constants | Software project organization | C++ user input | C++ header files | Cmake | C++ pointers | C++ data types | Control flow

## Object-Oriented Programming in C++

C++ polymorphism | Access specifiers | Encapsulation | C++ inheritance | Raii | Smart pointers | C++ header files | C++ pointers | C++ standard library | C++ data structures | Classes in programming | Errors and exceptions | Resource leaks | Stack memory | Intermediate object-oriented programming | Heap memory | Memory leaks | Dynamic memory allocation | Generic programming | C++ classes | C++ copy and move semantics

## Memory Management

Computer memory architecture | Dynamic memory allocation | C++ copy and move semantics | Heap memory | Memory management in c++ | R-values | Smart pointers | Buffer overruns | Memory-efficient programming | Call-by-value | Rule of three | Memory leaks | L-values | Code debugging | Segmentation fault | Automatic memory allocation | Hexadecimal numbers | Variable scope | Call-by-reference | Rule of five | Memory fragmentation | Stack memory | Ownership in memory management | Uninitialized memory bugs | Call stacks | Raii | Virtual memory

## Concurrency

Concurrent computing | Futures and promises | Message queues | Parallel computing | Data races | Threading

# Courses

## 01. Introduction to C++

14 hours

This course guides learners through the essentials of C++ programming. Starting with an overview and setup, students explore data structures using the Standard Template Library (STL), delving into functions and modular programming as well as Object-Oriented Programming (OOP). The course introduces fundamental algorithms through a hands-on search implementation, culminating in a practical project: building a Route Planner using OpenStreetMap. By the end, participants will be equipped with the skills to design and develop efficient C++ applications.

| | | |
|---|---|---|
| L1 | Welcome | Explore C++ with insights from its creator Bjarne Stroustrup, learn its industry impact, effective learning strategies, and key coding guidelines. |
| L2 | Introduction to C++ and Setup | Get started with C++: learn program structure, compilation, data types, input/output, expressions, conditionals, loops, and user interaction through hands-on coding exercises. |
| L3 | Working with Data Structures and STL | Learn to use arrays, vectors, and STL algorithms in C++ for data storage, string manipulation, file I/O, and efficient error handling to write robust, maintainable programs. |
| L4 | Functions, Modularity and OOP | Master C++ modularity by organizing code with functions, multi-file structure, and OOP—using classes, pointers, scope, linkage, and object lifecycle features for scalable design. |
| L5 | Introduction to Algorithms – A Search Implementation* | Learn to model search problems with graphs and grids, implement A* pathfinding in C++, use STL containers and lambdas, analyze complexity, and build projects with CMake. |
| L6 | Project:<br>Build an OpenStreetMap Route Planner | In this project, you will build a route planner that determines and visualizes the optimal path between two points using real-world map data from the OpenStreetMap project. |

## 02. Object-Oriented Programming in C++

12 hours

In this course, participants will explore fundamental OOP concepts and their applications. Starting with class design and encapsulation, the course progresses to inheritance and composition, helping students construct effective class hierarchies. Delving into advanced topics, the lessons cover polymorphism and templates, crucial for building flexible and reusable code. Additionally, learners will gain vital skills in error handling and debugging

techniques through targeted exercises. The course culminates in a practical project where students will develop a comprehensive system monitoring tool, solidifying their knowledge and coding competency.

| | | |
|---|---|---|
| L1 | Encapsulation and Class Design | Master encapsulation in object-oriented programming. Explore encapsulation principles, class structure, data hiding, and utilize getters and setters for efficient class design. |
| L2 | Inheritance, Composition, and Class Hierarchies | Learn C++ inheritance, composition, and class hierarchies: design clear, safe class structures using single/multiple inheritance, polymorphism, and the "is-a" vs. "has-a" approach. |
| L3 | Polymorphism in Depth and Templates | Master C++ polymorphism and templates to build flexible, efficient, and type-safe designs using abstract classes, interfaces, runtime dispatch, and the power of the standard library. |
| L4 | Error Handling Strategies and Debugging Techniques | Discover robust error handling in C++ with exceptions, return codes, and optionals, plus learn essential debugging, logging, assertions, and RAII for reliable, safe code. |
| L5 | Project:<br>Build a System Monitoring Tool | In this project, you will develop a simple system monitor program inspired by htop to track and display system metrics in real time. |

## 03. Memory Management

10 hours

This course provides a comprehensive exploration of how memory interacts with programming languages. Beginning with the fundamentals of system memory hierarchy, students learn essential concepts like pointers and pointer arithmetic. The course delves into dynamic memory allocation, highlighting ownership principles to manage memory effectively. Fundamental object-oriented topics such as copy constructors and move semantics are covered, addressing the crucial Rule of Three/Five for robust memory management. Additionally, advanced techniques involving smart pointers are introduced to enhance memory safety. The course culminates in a hands-on project to apply learned skills in real-world scenarios.

| | | |
|---|---|---|
| L1 | Memory Fundamentals and the System Memory Hierarchy | Explore C++ memory structure, pointers, addresses, the system memory hierarchy, caching, and GDB debugger use for efficient programming and troubleshooting. |
| L2 | Pointers and Pointer Arithmetic | Learn C++ pointers: how they store memory addresses, enable direct data manipulation, perform pointer arithmetic, and avoid common errors like null, dangling, and out-of-bounds pointers. |
| L3 | Dynamic Memory Allocation (Heap) and Ownership | Learn dynamic memory allocation in C++: using new/delete, preventing leaks, double deletion, and dangling pointers, and managing ownership with best practices and smart pointers. |

| | | |
|---|---|---|
| L4 | Copy Constructors, Move Semantics, and the Rule of Three/Five | Learn C++ copy constructors, move semantics, and the Rule of Three/Five. Master memory management, deep vs. shallow copy, and RAII for safe, efficient resource handling. |
| L5 | Smart Pointers and Advanced Memory Management | Master C++ smart pointers for safer, modern memory management: exclusive and shared ownership, avoiding leaks, breaking cycles, and refactoring legacy code using unique, shared, and weak pointers. |
| L6 | Project: Memory BOT | In this project, you will build a simple, terminal-based chatbot that creates a dialogue where users can ask questions about some aspects of memory management in C++ via a terminal. |

## 04. Concurrency

6 hours

This course provides a comprehensive introduction to concurrent programming concepts. It begins with an exploration of threads and parallel execution, focusing on how tasks can run simultaneously to enhance performance. The course then delves into critical aspects of shared data management and task synchronization, ensuring that multiple threads can operate without conflict. Students will learn about essential tools such as mutexes, locks, and condition variables, which are crucial for managing access to shared resources safely. Finally, participants will apply their knowledge in a project that integrates all learned concepts, fostering practical experience in designing and implementing concurrent systems.

| | | |
|---|---|---|
| L1 | Threads and Parallel Execution | Learn how to create, manage, and synchronize threads in C++ to unlock parallel execution, avoid race conditions, and ensure safe resource management using RAII and synchronization primitives. |
| L2 | Threads: Sharing Data and Task Synchronization | Master thread communication and synchronization using promises, futures, std::async, atomics, and lock-free programming for scalable, efficient concurrent applications. |
| L3 | Mutexes, Locks, and Condition Variables | Master advanced thread synchronization with mutexes, locks, and condition variables to build correct, efficient, and deadlock-free concurrent systems using modern C++ techniques. |
| L4 | Project: Program a Concurrent Traffic Simulation | In this project, students will create a multithreaded traffic simulation where vehicles navigate intersections safely by using mutexes and synchronization to prevent collisions. |

# Meet Your Instructors

## Ishan Gosain

Engineering Manager at Rivian and VW Technology Group

Ishan is an engineering manager at Rivian and VW Technology Group where he specializes in Imaging, Camera and Edge AI applications for automotive. He has worked on software validation and integration of visual camera features, security features, and voice assistant features. He formerly worked at John Deere on the Perception team. He graduated from the Pennsylvania State University.

## Kedar Kodgire

Data and Cloud Engineering Lead

Kedar is a Data and Cloud Engineering Lead with a passion for designing scalable data systems and AI-powered cloud solutions. With deep expertise in platforms like Azure, GCP, Databricks, and Delta Lake, he specializes in building robust data pipelines that turn complexity into clarity. As a technical educator, he's dedicated to making advanced concepts approachable—helping learners bridge the gap between theory and realworld application.

## Aspen Olmsted

Associate Professor of Computer Science

Aspen is an associate professor of computer science at Wentworth Institute of Technology in Boston, MA. He obtained a Ph.D. in Computer Science and Engineering from the University of South Carolina. Before embarking on his academic career, he served as CEO of Alliance Software Corporation. Alliance Software developed N-Tier enterprise applications for the performing arts and humanities market. Dr. Olmsted's research focus is on developing algorithms and architectures for distributed enterprise solutions that ensure security and correctness while maintaining high availability. In his Secure Software Development Lab, Aspen mentors over a dozen graduate and undergraduate students each year.

## Andrew Sanford

Cybersecurity Engineer

Andrew brings over a decade of expertise across cybersecurity, software development, and artificial intelligence. With proficiency in secure code development built over many years, he has contributed to projects ranging from secure application design to code analysis and vulnerability management. His work has included oversight of and running bug bounty programs and implementation of best practices in secure coding. His experience in AI includes the development of predictive systems and the integration of machine learning tools in operational settings.

# Why Udacity

## Demonstrate proficiency with practical projects

Projects are based on real-world scenarios and challenges, allowing you to apply the skills you learn to practical situations, while giving you real hands-on experience

- ✓ Gain proven experience
- ✓ Retain knowledge longer
- ✓ Apply new skills immediately

## 24/7 access to real human support

Reviewers provide timely and constructive feedback on your project submissions, highlighting areas of improvement and offering practical tips to enhance your work

- ✓ Get help from subject matter experts
- ✓ Gain valuable insights and improve your skills
- ✓ Learn industry best practices